

Лекция 3

Tool calling

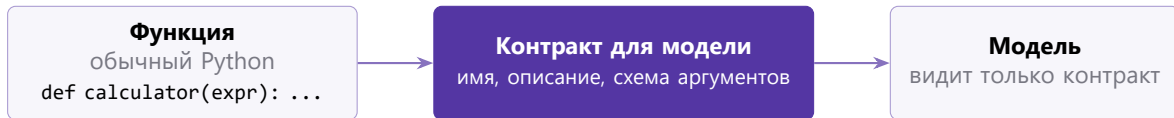
как модель зовёт функцию, что ей вернуть и что делать, когда функция сломалась

- ❶ **Механика**: схема, вызов, результат, полный цикл в коде
- ❷ **Описание как промпт**: от чего зависит, какой инструмент выберет модель
- ❸ **Ошибки и пустота**: как возвращать сбои, чтобы агент не падал и не врал
- ❹ **Безопасность и набор**: сколько инструментов давать и какие права

Важная мысль

Модель не вызывает функции. Она пишет структурированный запрос «вызови это с такими аргументами». Вызывает ваш код. Вся ответственность за то, что произошло, на нём.

Инструмент это функция плюс контракт



- Модель никогда не видит код функции. Она видит имя, описание и JSON Schema параметров
- Значит, качество выбора инструмента зависит от **текста контракта**, а не от качества кода
- Один и тот же контракт можно подложить под разные реализации: настоящий поиск, заглушку, тест

Контракт в API: параметр tools

```
tools = [{
  "type": "function",
  "function": {
    "name": "calculator",
    "description": "Считает арифметическое выражение над известными числами, "
                  "например «3400 - 1290» или «45000 * 0.2». Не ищет данные.",
    "parameters": {
      "type": "object",
      "properties": {"expr": {"type": "string",
                              "description": "выражение на Python-синтаксисе"}},
      "required": ["expr"]
    }
  }
}]
```

- Формат JSON Schema: типы, обязательные поля, перечисления, вложенные объекты
- Описание параметра тоже читает модель. «expr: string» хуже, чем «выражение на Python-синтаксисе»

Что возвращает модель, когда хочет инструмент

```
{
  "role": "assistant",
  "content": "Посчитаю разницу.",
  "tool_calls": [{
    "id": "call_8f2a",
    "type": "function",
    "function": {"name": "calculator", "arguments": "{\"expr\": \"3400 - 1290\"}"}}
  ]
}
```

- content может быть текстом, пустой строкой или null. Мысль перед действием живёт здесь
- arguments это **строка с JSON**, а не объект. Её надо распарсить, и парсинг может упасть
- id обязателен: по нему модель потом свяжет результат с вызовом
- Признак «модель хочет инструмент» это непустой tool_calls, а не текст «вызываю калькулятор»

Возвращаем результат

```
messages.append(assistant_message)                # сначала ответ модели как есть

for call in assistant_message["tool_calls"]:
    args = json.loads(call["function"]["arguments"])
    result = TOOLS[call["function"]["name"]](**args)
    messages.append({
        "role": "tool",
        "tool_call_id": call["id"],                # тот самый id
        "content": str(result),                    # всегда строка
    })
```

- Порядок обязателен: сообщение ассистента с вызовами, потом результат **на каждый** вызов
- Если вызовов три, результатов должно быть три, все до следующего обращения к модели
- Пропустили один результат: API вернёт ошибку. Вернули не строкой: тоже

Так не надо

flights: «Рейсы.»

hotels: «Отели.»

calc: «Считает.»

Модель гадает, куда идти с вопросом про отмену брони: в рейсы, в отели или в калькулятор.

Три правила: что делает, **чего не делает**, чем отличается от похожих инструментов. Проверим на следующем слайде.

Так надо

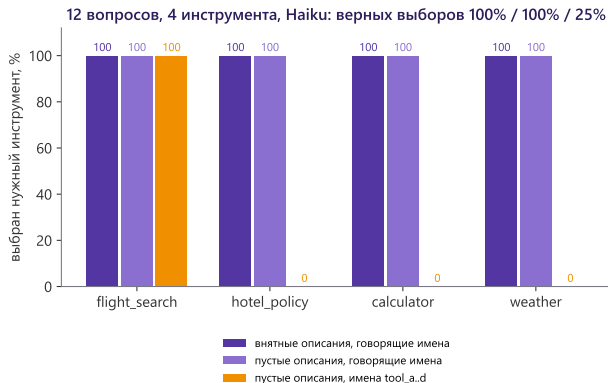
flight_search: «Поиск авиарейсов и цен между городами на даты.»

hotel_policy: «Правила отелей: заезд, отмена брони, питание, животные.»

calculator: «Арифметика над уже известными числами. Не ищет данные.»

Сказано не только что делает, но и чего не делает и чем отличается от соседа.

Эксперимент: те же вопросы, разные описания



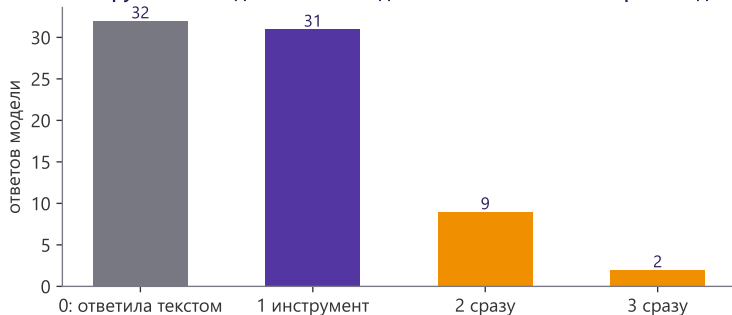
- Двенадцать вопросов, температура ноль, одна модель, меняется только контракт
- Говорящее имя даёт 100% даже с пустым описанием. Уберите и имя, и описание, и модель **сваливает всё в первый инструмент**: контракт это имя плюс описание, и оба читает модель

- **Типы**: string, integer, number, boolean, array, object
- **enum**: закрытый список значений.
Месяцы, категории, режимы
- **required**: без него модель пропустит поле, которое вам нужно
- **description** у каждого поля: формат, единицы, примеры
- Строгий режим у провайдера:
гарантия, что аргументы валидны по схеме

Мало параметров, простые типы

Один инструмент с одним строковым аргументом модель вызывает почти без ошибок. Инструмент с восемью вложенными полями путает половину. Лучше три простых инструмента, чем один универсальный.

Сколько инструментов модель зовёт за один ответ: 74 ответов из трейсов демо-агента



- Модель может попросить несколько инструментов в одном ответе: две высоты двумя поисками сразу
- Ваш код обязан выполнить все и вернуть все результаты одним пакетом, порядок сохранить
- Выполнять можно параллельно, если инструменты независимы. Экономия времени, не денег

Ошибка инструмента это данные, а не исключение

```
def execute(call):
    name = call["function"]["name"]
    try:
        args = json.loads(call["function"]["arguments"])
        return str(TOOLS[name](**args))
    except json.JSONDecodeError:
        return "ошибка: аргументы не JSON"
    except KeyError:
        return f"ошибка: инструмента {name} нет"
    except Exception as e:
        return f"ошибка инструмента: {e}"
```

- Исключение внутри инструмента не должно ронять цикл
- Текст ошибки возвращается модели как наблюдение: она меняет план
- Внятная ошибка лучше короткой: «нет колонки month, есть month_name» модель починит сама

- Всегда строка, но внутри может быть JSON, таблица или текст
- Короткие факты текстом: «330 м со шпилем»
- Структуры JSON-ом: список заказов, строки таблицы. Модель читает JSON лучше, чем «красивый» вывод
- Источник в начале: [dostavka.md], [Википедия: Эйфелева башня]. Потом агент сможет сослаться
- Единицы измерения и даты явно: «540,1 м», «за июнь 2026»

Длина наблюдения

Страница текста в каждом наблюдении это тысяча токенов на каждом следующем шаге. Возвращайте самое релевантное: первые N предложений, топ-k строк, сниппеты вокруг совпадений. Если обрезали, скажите об этом в самом наблюдении.

Пустой результат это не ошибка, и их нельзя путать

Инструмент сломался

«ошибка: таймаут поиска»

Модель должна **повторить или обойти**:
другой инструмент, другой запрос.

Данных нет

«ничего не найдено» или «записей за июль нет»

Модель должна **честно сказать** об этом,
а не искать в десятый раз.

- Если инструмент на оба случая возвращает пустую строку, модель не отличит одно от другого и будет либо крутиться, либо выдумывать
- Пустой результат стоит формулировать так, чтобы он читался человеком: «записей нет: данные за июль ещё не загружены»
- Это первое место, где рождаются галлюцинации агента. Второе, промпт, мы разбирали на прошлой лекции

```
import ast, operator as op
OPS = {ast.Add: op.add, ast.Sub: op.sub,
       ast.Mult: op.mul, ast.Div: op.truediv}

def ev(node):
    if isinstance(node, ast.Constant):
        return node.value
    if isinstance(node, ast.BinOp):
        return OPS[type(node.op)](ev(node.left),
                                   ev(node.right))
    raise ValueError("недопустимое выражение")

def calculator(expr):
    return str(ev(ast.parse(expr, mode="eval").body))
```

- eval выполнит любую строку, которую напишет модель. А модель пишет то, что прочитала в документах
- Разбор через ast: разрешены только числа и четыре операции, всё остальное ошибка
- Тот же принцип для любого инструмента: **белый список** того, что можно, а не чёрный список того, что нельзя

Зачем

- Задачи, где нужен настоящий Python: таблицы, даты, парсинг, статистика
- Один инструмент закрывает сотню специализированных
- На GSM8K агент с `python_exec` обгоняет агента с калькулятором

Как не пострадать

- Отдельный процесс с таймаутом 5–10 секунд
- Без сети и без доступа к вашей файловой системе
- Лимит памяти, лимит длины вывода
- В идеале контейнер или песочница. На Kaggle сам ноутбук и есть одноразовая машина

Возвращать модели `stdout`, `stderr` и код выхода. `Traceback` это тоже данные: модель починит код сама, если увидит ошибку.

Пример: поиск изнутри

```
def web_search(query: str) -> str:
    hits = wiki_search(query, limit=1)          # 1. найти статью
    if not hits:
        return "ничего не найдено"              # пусто, а не ошибка
    text = wiki_fulltext(hits[0]["title"])       # 2. полный текст
    sents = split_sentences(text)
    best = top_by_overlap(sents, query, k=4)     # 3. сниппеты под запрос
    return f"[Википедия: {hits[0]['title']}] {sents[0]} ... {' ... '.join(best)}"
```

- Найти, достать, отобразить: три этапа, и модель видит только третий
- Первая версия возвращала вступление статьи. Высоты Эйфелевой башни во вступлении нет, и агент честно сдался. Сниппеты под запрос починили это без единого изменения в промпте
- **Качество агента часто живёт внутри инструмента**, а не в модели и не в промпте

Тип	Примеры	Правило
Только чтение	поиск, база знаний, статус заказа	можно давать свободно
Вычисление	калькулятор, код в песочнице	можно, если изолировано
Побочный эффект	письмо, платёж, запись в базу, удаление	подтверждение человеком

- Агент-читатель не должен иметь права писать. Не потому что модель злая, а потому что она читает документы, в которых может быть чужая инструкция
- Опасное действие: инструмент возвращает «требуется подтверждения», человек нажимает кнопку, только тогда выполняется
- На пятой неделе покажем, как через документ агента заставляют отправить письмо. Защита начинается здесь, с прав

- Каждая схема это токены в **каждом** вызове: десять инструментов по 150 токенов это 1500 токенов префикса
- Чем больше похожих инструментов, тем чаще модель путает. Пять хорошо, пятнадцать плохо
- Инструменты можно менять от шага к шагу: на этапе поиска одни, на этапе ответа другие
- Если инструментов десятки: поиск по инструментам, модель сначала находит нужный, потом зовёт

Ориентир для первой недели

Три инструмента: калькулятор, поиск, исполнение кода. Этого хватает на GSM8K и HotpotQA, и модель не путается.

Значение	Что делает модель	Когда нужно
auto	сама решает, звать инструмент или ответить	агент, по умолчанию
required	обязана вызвать какой-нибудь инструмент	первый шаг без ответа «из головы»
конкретное имя	обязана вызвать именно этот	structured output через инструмент
none	инструменты видны, но звать нельзя	финальный ответ после сбора данных

- Structured output из первой лекции и tool calling это одна механика: схема, по которой модель обязана сгенерировать JSON
- Приём: последний шаг агента с none, чтобы модель точно ответила текстом, а не ушла на новый круг

- Модель может повторить вызов: не увидела результат, решила перепроверить, зациклилась
- Для чтения это лишние деньги. Для записи это **два письма клиенту** или два списания
- Инструменты с побочным эффектом делайте идемпотентными: повторный вызов с теми же аргументами ничего не меняет

Как

Ключ операции из аргументов, проверка «уже делали» перед выполнением, кэш результатов чтения на время прогона. Детектор повторов из прошлой лекции работает поверх этого, а не вместо.

- Инструмент это обычная функция, у неё должны быть обычные тесты
- Проверять граничные случаи: пустой запрос, кириллица, огромное число, таймаут
- Проверять формат ошибок: модель получит строку, и строка должна быть понятной
- Отдельно: набор «какой инструмент на какой вопрос», прогон на модели, доля верных выборов

Зачем разделять

Когда агент ошибся, есть два подозреваемых: модель выбрала не то или инструмент вернул не то. Тесты инструмента снимают второго подозреваемого за секунду.

- ❶ Описание «Рейсы.» и удивление, что модель ищет там погоду и правила отелей
- ❷ Пустой результат и ошибка возвращаются одинаково: модель не отличает «нет данных» от «сломалось»
- ❸ Исключение из инструмента роняет весь цикл
- ❹ Результат не на каждый вызов или не строкой: ошибка API
- ❺ `eval`, `os.system`, запросы к базе из строки модели без проверки
- ❻ Инструмент с побочным эффектом без подтверждения и без идемпотентности
- ❼ Пятнадцать инструментов «на всякий случай»: дорого и путается

Все семь встречаются в реальных репозиториях. Три из них в популярных туториалах.

Руками

- Три инструмента для ReAct-агента: `calculator` через `ast`, `web_search` через Tavily или DuckDuckGo, `python_exec` в отдельном процессе
- Обработка ошибок и пустых результатов
- Прогон на GSM8K и HotpotQA, сравнение агента с калькулятором и с исполнением кода

Следующая неделя про память: как не тащить всю историю в каждый вызов и как агент вспоминает вас между разговорами.

Подготовить

Ключ Tavily, бесплатной тысячи запросов в месяц хватит.
Репозиторий для агента: код в файлах, не в ячейках ноутбука.

- 1 Модель не вызывает функции. Она пишет запрос на вызов, а выполняет ваш код. Ответственность на нём
- 2 Модель видит только контракт: имя, описание, схему. Описание это промпт, и оно решает, куда пойдёт вопрос
- 3 Ошибка инструмента возвращается моделью как текст, а не роняет цикл. Пустой результат и ошибка обязаны различаться
- 4 Права важнее промпта: читать можно свободно, действовать только через подтверждение и идемпотентно
- 5 Три простых инструмента лучше одного универсального. Тестируйте их отдельно от модели

Вопросы?

следующая лекция: память агента, контекстное окно как бюджет и что агент помнит между разговорами